ESP32–ThingSpeak TPI 4K3: Guía técnica

Esta guía explica la implementación del TPI con dos sketches, su variante opcional de recuperación por OTA, cómo se integran sus componentes y qué aspectos se verificaron y cuáles siguen pendientes. Está pensada para compañeros que necesiten entender, compilar, probar o compartir el proyecto de forma segura, sin exponer credenciales.

> **Advertencia sobre credenciales — leer antes de compartir.** Las copias locales actuales de los sketches originales de envío y lectura contienen credenciales reales de Wi-Fi, y la copia del emisor también contiene una clave de escritura de ThingSpeak. **No compartas, subas, confirmes en Git, captures en pantalla ni pegues esas copias fuente tal como están.** Reemplazá las credenciales locales por marcadores neutros antes de compartir; rotá cualquier credencial que se haya expuesto fuera de su entorno privado previsto. Esta guía no contiene credenciales reales, SSID privados ni direcciones IP privadas.
>
> **Sobre esta copia.** Es la guía del grupo 1 tal cual, publicada sin claves: en los sketches del ZIP la red, la clave de escritura y el canal van con marcadores (`TU_RED_WIFI`, `TU_API_KEY`, `1234567`). Los sketches ya se llaman `sketch_envio_grupo_1` y `sketch_lectura_grupo_1` (lo que dice de «XX» es de antes de la entrega). La carpeta `wokwi/`, la variante del lector con OTA y el registro de tareas que se nombran acá no vienen en el ZIP.

Ruta rápida

1. **Empezá por el modelo del trabajo:** cargá el Sketch 1 por USB; lee sensores, controla los LED, publica seis valores y anuncia ArduinoOTA.
2. **Usá OTA para la transición habitual:** cargá el Sketch 2 original desde el puerto de red detectado por el IDE; lee los últimos valores del canal público de ThingSpeak y los muestra en pantalla.
3. **Usá la variante opcional de recuperación solo cuando haga falta:** su servicio ArduinoOTA adicional permite volver a cargar el Sketch 1 en la placa por red. La dirección de retorno compiló, pero **no se verificó físicamente**.
4. **Prepará el entorno local:** instalá el core Arduino para ESP32 y las bibliotecas externas de `wokwi/libraries.txt`; ingresá las credenciales solo en copias locales privadas.
5. **Diferenciá los tipos de evidencia:** el comportamiento en Wokwi, las comprobaciones de código/compilación y las pruebas OTA físicas son evidencias distintas. Consultá Verificación y nivel de confianza.

1. Contexto del proyecto y del trabajo práctico

El TPI solicita dos programas complementarios para una ESP32 y un canal público de ThingSpeak:

- **Sketch 1 — emisor/controlador:** se carga inicialmente por USB. Lee el DHT22 y el potenciómetro, controla dos salidas PWM, ajusta un LED externo con dos entradas táctiles capacitivas, escribe seis campos en el canal periódicamente y habilita OTA para cargar el Sketch 2 por Wi-Fi.
- **Sketch 2 — lector/pantalla:** se carga mediante el punto OTA que inicia el Sketch 1. Lee la última entrada del canal, cuenta las pulsaciones del encoder y alterna los seis valores en dos páginas.
- **Entrega:** los dos archivos fuente con el número de grupo real en los nombres requeridos, más un archivo de texto con los integrantes y los datos del canal. La clave de escritura debe quedar únicamente en el artefacto privado de entrega que exige la materia, no en un repositorio público ni en esta guía.

También hay una **variante del lector con OTA habilitado**. Conserva el comportamiento del Sketch 2 y agrega un servicio OTA. Sirve como puente de recuperación para volver al Sketch 1; no reemplaza los dos sketches principales pedidos.

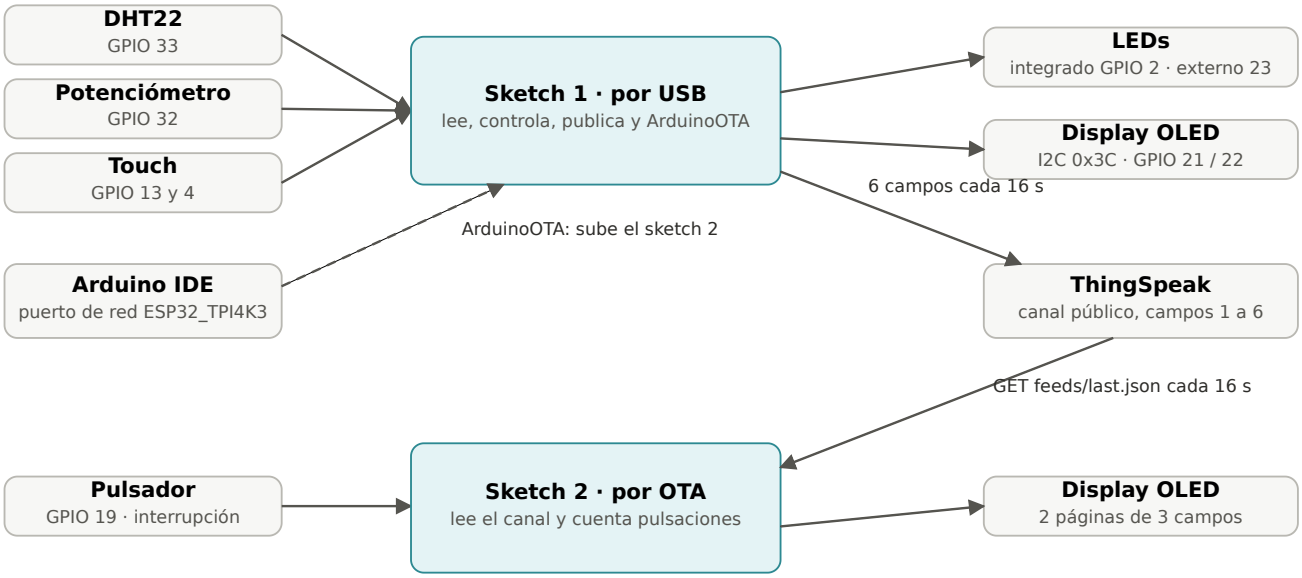
Mapa de requisitos a código

Requisito del trabajo / proyecto	Implementación	Notas y límites
El brillo del LED integrado sigue al potenciómetro; mostrar el valor PWM de 12 bits	Emisor: `updatePotentiometerLed()`, `drawStatus()`	GPIO2 es una suposición basada en el ejemplo del docente, no una garantía para todas las placas ESP32.
Mostrar temperatura y humedad	Emisor: `updateSensorReadings()`, `drawStatus()`	El muestreo del DHT22 se limita a un ciclo de lectura cada 2 segundos; si una lectura falla, se conservan los últimos valores válidos.
Aumentar/disminuir el brillo del LED externo con dos entradas táctiles; mostrar porcentaje	Emisor: `updateTouchControls()`, `applyExternalLedBrightness()`, `drawStatus()`	Las entradas físicas son GPIO13/GPIO4. En Wokwi se emulan con pulsadores en GPIO25/GPIO26 en una configuración exclusiva para Wokwi.
Publicar seis campos cada 16 segundos	Emisor: `sendThingSpeakUpdate()` y `loop()`	Se omite una escritura si no hay Wi-Fi o datos válidos del DHT.
Cargar el Sketch 1 por USB y habilitar OTA para el Sketch 2	Emisor: `connectToWiFi()`, `startOtaWhenConnected()`, `loop()`	El usuario confirmó físicamente USB → Sketch 2 original por OTA.
Leer los últimos valores del mismo canal	Lector y lector OTA: `readThingSpeak()`, `parseChannelResponse()`	El canal es público, por eso la clave de lectura es opcional y permanece comentada.
Contar pulsaciones y mostrar el contador con campos legibles	Lector y lector OTA: `onButtonPressed()`, `getButtonPressCount()`, `drawCurrentPage()`	Solo se usa el pulsador del encoder; estos sketches no usan sus contactos de giro.
La variante OTA del Sketch 2 puede restaurar el Sketch 1	Lector OTA opcional: `startArduinoOTA()`	Compiló; todavía no se demostró la carga de retorno en la placa física.
Tolerar la ausencia de OLED durante pruebas OTA con la placa sola	Los tres sketches: inicialización de pantalla protegida y funciones de dibujo	Permite iniciar en modo sin pantalla; no verifica el hardware del sensor, touch, LED, encoder ni display.
Proveer la configuración de simulación	`wokwi/diagram.json`, `wokwi/libraries.txt`, `wokwi/README.md`	La simulación representa parte del hardware de otra manera; no demuestra el funcionamiento en una placa física.

Aceptación funcional en pocas palabras

- **Requisitos centrales del TPI:** los dos sketches principales implementan las funciones de sensor/control/publicación y lectura/pantalla descritas en el PDF.
- **La entrega UV todavía no está lista:** los nombres contienen `XX`; todavía faltan el número de grupo, la lista de integrantes y el archivo de texto requerido.
- **Todavía falta completar la validación de hardware:** siguen pendientes la carga OTA de retorno y las comprobaciones físicas de los componentes. Antes de decir que todo el sistema está validado en hardware, consultá la tabla de verificación.

2. Modelo del sistema y las comunicaciones



Hay dos usos de red diferentes: el tráfico de ThingSpeak es transferencia de datos de la aplicación por Wi-Fi; ArduinoOTA es una vía local de transferencia de firmware entre el IDE y la ESP32. Wokwi provee una ESP32 simulada y componentes virtuales; no reproduce la ruta de red OTA de la placa física.

Flujos de punta a punta

Flujo de datos: sensores y controles → estado del emisor → actualización de seis campos en ThingSpeak → JSON público de la última entrada → el lector analiza los valores → dos páginas en OLED. El contador del pulsador es local al lector y no es un campo de ThingSpeak.

Flujo de firmware: cargar el Sketch 1 por USB → la ESP32 se conecta a Wi-Fi e inicia ArduinoOTA → el IDE detecta un puerto de carga de red → cargar el Sketch 2 por OTA. Para recuperarse, primero cargá la variante del lector con OTA habilitado; cuando su servicio OTA esté listo, seleccioná el mismo punto de red de la ESP32 y cargá el Sketch 1.

3. Función de cada herramienta y biblioteca

Plataforma y herramientas de desarrollo

Herramienta / componente	Función en este proyecto
ESP32	Microcontrolador. Lee entradas digitales/analógicas/táctiles, genera PWM y usa I2C, ejecuta los sketches y se conecta a Wi-Fi. El LED integrado y la compatibilidad táctil exacta dependen de la placa y su revisión.
Arduino IDE	Abre el sketch, selecciona la placa ESP32 y el puerto USB/de red, compila con el core ESP32 seleccionado y carga el firmware. El Monitor Serie muestra diagnósticos a 115200 baudios.
Core Arduino para ESP32	Paquete de compatibilidad Arduino y soporte de placas ESP32. Provee `WiFi`, `Wire`, `HTTPClient`, `ArduinoOTA`, GPIO, `touchRead`, temporización, interrupciones y las API PWM específicas de ESP32 usadas aquí. Debe seleccionarse como plataforma de placa al compilar.
ArduinoOTA	Biblioteca del core ESP32 que permite cargar firmware por la red local. El sketch la inicia solo después de conectar Wi-Fi y llama a `ArduinoOTA.handle()` repetidamente desde `loop()`. Estas fuentes no configuran una contraseña OTA.

Herramienta / componente	Función en este proyecto
Wi-Fi	Conecta la placa a una red en modo estación. Transporta el tráfico HTTP de ThingSpeak y la comunicación OTA local. Para detectar y cargar por OTA, la laptop y la placa necesitan una ruta de red entre sí.
HTTPClient	Cliente HTTP del core ESP32 que los lectores usan para hacer un GET al endpoint JSON de la última entrada de ThingSpeak.
ThingSpeak	Servicio de canal en la nube que almacena campos con marca temporal. El Sketch 1 escribe una entrada y ambos lectores consultan la más reciente. El canal configurado tiene seis campos.
Wokwi	Simulador de circuitos y firmware en el navegador. Sirve para ejercitar lógica, disposición en pantalla, respuesta de entradas y conexión con ThingSpeak, pero sus modelos no son idénticos a todo el hardware físico.
Monitor Serie	Consola de diagnóstico. Los sketches informan el estado de Wi-Fi, sensor, HTTP y OTA. Configuralo a **115200 baudios**. Es especialmente útil cuando faltan la OLED u otros periféricos.
Arduino Cloud Editor (opcional)	Podría usarse como editor/entorno de compilación si allí está disponible el paquete de placa ESP32. No es obligatorio para el TPI, y su OTA administrado en la nube es distinto de ArduinoOTA en este proyecto.

Bibliotecas externas

La lista de bibliotecas de Wokwi es la referencia de dependencias externas. `WiFi`, `Wire`, `HTTPClient` y `ArduinoOTA` vienen con el core Arduino para ESP32, no se agregan como bibliotecas Arduino separadas.

Biblioteca externa	Usada por	Propósito
Adafruit GFX Library	Todos los sketches (a través de la API de la biblioteca de pantalla)	Primitivas gráficas y de texto comunes que usa el controlador OLED.
Adafruit SH110X	Todos los sketches	Controlador para pantallas de la familia SH1106; construye la pantalla 128×64 y envía texto/cuadros por I2C.
DHT sensor library	Emisor	Lee la temperatura y la humedad relativa del DHT22.
Adafruit Unified Sensor	Emisor / dependencia de DHT	Interfaces y tipos de sensor compartidos por las bibliotecas Adafruit; incluila explícitamente como dependencia.
ThingSpeak	Emisor	Provee `ThingSpeak.begin()`, `setField()` y `writeFields()` para actualizar el canal.
ArduinoJson	Ambos lectores	Analiza el JSON que devuelve ThingSpeak y obtiene los seis valores `fieldN`.

Preparación local

1. Instalá Arduino IDE (u otro entorno de compilación compatible con ESP32) y agregá el **core Arduino Espressif ESP32** mediante el flujo de Boards Manager.
2. Seleccioná el objetivo ESP32 DevKit V1 / la placa ESP32 que coincida con el hardware. El nombre exacto del menú puede variar según la versión del core.
3. Instalá las bibliotecas externas enumeradas en `wokwi/libraries.txt` mediante Arduino Library Manager, incluida la dependencia transitiva Unified Sensor.
4. Abrí el archivo `.ino` dentro de la carpeta de sketch con el mismo nombre. Compilá el emisor y el lector como sketches separados; no combines sus funciones `setup()`/`loop()`.

5. Configurá las credenciales **solo en copias locales privadas**. Usá marcadores neutros en cualquier copia compartida. El Sketch 1 necesita Wi-Fi y una clave de escritura ThingSpeak para publicar; el lector normal necesita Wi-Fi y ninguna clave si el canal es público. La variante de lector OTA usa Wi-Fi, pero deliberadamente no tiene clave de escritura.
6. Para cargar por USB, seleccioná el puerto serie. Para OTA, primero iniciá un sketch que arranque ArduinoOTA, esperá a que se conecte y luego seleccioná en el IDE el puerto de red detectado.
7. Abrí el Monitor Serie a **115200 baudios** para consultar mensajes de conexión y errores.

El proyecto de Wokwi tiene su propio `libraries.txt` y `diagram.json`; copiá el contenido en las pestañas correspondientes del proyecto Wokwi. No copies credenciales activas a un proyecto Wokwi público ni que se muestre en pantalla. Si una simulación privada necesita una clave, mantené privado ese proyecto y quitá la clave antes de compartirlo.

4. Mapa de pines y supuestos de cableado

Función	GPIO / interfaz	Emisor	Lector / lector OTA	Notas
Datos del DHT22	GPIO33	Sí	No	El mapeo del DHT22 sigue las indicaciones del curso/docente.
Cursor del potenciómetro	GPIO32 / ADC	Sí	No	Se lee como 0–4095 luego de configurar ADC a 12 bits. Conectá los extremos a 3V3 y GND según corresponda.
LED integrado	GPIO2	Sí	No	Suposición del ejemplo de LED integrado del curso; verificá la placa real. El pin del LED integrado puede ser otro o no existir.
PWM de LED externo	GPIO23	Sí	No	El diagrama Wokwi incluye una resistencia de 220 Ω en serie. En hardware físico usá una limitación de corriente adecuada.
Aumentar con touch	GPIO13	Sí	No	Entrada táctil capacitiva en ESP32 física; el umbral depende de la placa y el entorno.
Disminuir con touch	GPIO4	Sí	No	Tiene la misma salvedad de calibración que GPIO13.
Pulsador Wokwi para aumentar touch	GPIO25	Solo copia Wokwi	No	Se usa únicamente cuando `TOUCH_SIMULATION_MODE` está habilitado en la copia de simulación.
Pulsador Wokwi para disminuir touch	GPIO26	Solo copia Wokwi	No	No lo uses en lugar de GPIO13/GPIO4 capacitivos físicos en el emisor entregado.
SDA / SCL de OLED	GPIO21 / GPIO22, I2C	Sí	Sí	Dirección `0x3C`; las fuentes físicas usan un controlador de la familia SH1106.
A / B del encoder	GPIO18 / GPIO5	No	No se leen	Están en el diagrama Wokwi, pero la lógica actual del lector no usa el giro.
Pulsador del encoder	GPIO19	No	Sí	`INPUT_PULLUP`, supuesto activo en nivel bajo e interrupción por flanco descendente.
Alimentación/masa común	3V3 / GND	Según necesidad	Según necesidad	Respetá las especificaciones de los módulos reales y el cableado de la placa del curso.

Calibración táctil y PWM de 12 bits

El emisor configura las lecturas ADC a 12 bits y la resolución PWM **por pin de salida** para GPIO2 y GPIO23. Esto coincide con la API del core Arduino ESP32 que usa el proyecto. Con 12 bits, el rango PWM nominal es `0..4095`; el valor bruto mostrado para el LED integrado no es un porcentaje.

El LED externo cambia en pasos nominales de 5 %: cada evento táctil ajusta el PWM en `PWM_MAX / 20`, con saturación en cero o en el máximo. El porcentaje mostrado se calcula mapeando el rango PWM a `0..100`.

Las lecturas táctiles físicas se registran periódicamente. El código supone que al tocar suele disminuir el valor de `touchRead()`. Los umbrales **950** y **850** son valores iniciales del ejemplo del docente, no valores de calibración garantizados. Primero inspeccioná en el Monitor Serie los valores en reposo y al tocar en la placa objetivo; después ajustá cada umbral y el margen de liberación según la placa, el electrodo/cable, la conexión a tierra y el entorno. La lógica de bloqueo/histéresis evita repetir pasos mientras se mantiene apoyado el dedo. En Wokwi, los botones emulan una pulsación, no miden capacitancia.

5. Contrato de datos y temporización de ThingSpeak

El Sketch 1 escribe una entrada con seis campos. Los índices y significados están definidos por el TPI y se implementan en `sendThingSpeakUpdate()`.

Campo ThingSpeak	Significado	Origen / conversión
`field1`	Entero aleatorio, rango inclusivo 100–500	`random(100, 501)`; en la API Arduino el límite superior no está incluido.
`field2`	Temperatura en °C	Última temperatura válida del DHT22 (`float`).
`field3`	Humedad relativa en porcentaje	Última humedad válida del DHT22 (`float`).
`field4`	Segundos transcurridos desde el inicio	`millis() / 1000`; vuelve a cero al reiniciar y eventualmente desborda según el tipo del temporizador.
`field5`	Valor PWM del LED integrado	Valor de 12 bits `0..4095`, leído del potenciómetro y aplicado a GPIO2.
`field6`	Porcentaje de brillo del LED externo	PWM mapeado de `0..4095` a `0..100`.

Frecuencia: el emisor intenta actualizar cada **16,000 ms**. Después de la lectura inicial en `setup()`, las lecturas periódicas del DHT se programan cada **2,000 ms** de tiempo transcurrido; la pantalla se actualiza cada **250 ms** y se intenta reconectar Wi-Fi aproximadamente cada **10,000 ms**. El primer intento de ThingSpeak se programa cuando transcurre el intervalo de 16 segundos del loop; no se hace una escritura inmediata en setup. Las operaciones de red y el inicio pueden agregar demoras.

Para intentar una escritura, el emisor requiere una estación Wi-Fi conectada y lecturas válidas de temperatura y humedad. Imprime el código de respuesta de ThingSpeak; el resultado HTTP/de biblioteca `200` se considera aceptado. Si falla una lectura del sensor, se conservan los últimos valores válidos; si nunca se obtuvo una lectura válida, se omiten las escrituras. Aunque se omita un intento, se actualiza el temporizador de envío del loop, por lo que el siguiente intento ocurre según el intervalo normal, sin reintentos continuos.

El lector solicita `http://api.thingspeak.com/channels/<channel-id>/feeds/last.json`, analiza `field1` a `field6` y luego actualiza los valores de pantalla solo si el JSON se analiza correctamente y están las seis claves. La declaración opcional de clave de lectura y su parámetro en la URL están comentados porque el canal es público. Si el canal pasa a ser privado, configurá una clave de lectura de forma privada y habilitá el parámetro correspondiente.

6. Recorrido del Sketch 1 — emisor, controles y OTA

Fuente: `sketch_envio_grupo_XX/sketch_envio_grupo_XX.ino`

Estado y constantes globales

Modo de compilación, credenciales, canal y pines

Nombre	Tipo / valor inicial	Propósito
`TOUCH_SIMULATION_MODE`	Macro, `0`	Selecciona el modo táctil capacitivo físico. Ponelo en `1` **solo en una copia descartable para Wokwi**, donde los botones usan GPIO25/GPIO26.
`WIFI_SSID`	`const char *`	Nombre de la red local. Reemplazalo solo en una copia privada; nunca publiques el valor real.
`WIFI_PASSWORD`	`const char *`	Contraseña de la red local. Tratala como un secreto.
`THINGSPEAK_WRITE_API_KEY`	`const char *`	Clave que autoriza escrituras en el canal. Es secreta; no hace falta para OTA.
`THINGSPEAK_CHANNEL_ID`	`unsigned long`, identificador de canal configurado	Selecciona el canal de destino.
`DHT_PIN`	`uint8_t`, GPIO33	Pin de datos del DHT22.
`POT_PIN`	`uint8_t`, GPIO32	Entrada analógica del potenciómetro.
`INTEGRATED_LED_PIN`	`uint8_t`, GPIO2	Salida PWM; su uso como LED integrado es un supuesto específico de la placa.
`EXTERNAL_LED_PIN`	`uint8_t`, GPIO23	Salida PWM del LED externo.
`TOUCH_UP_PIN` / `TOUCH_DOWN_PIN`	`uint8_t`, GPIO13 / GPIO4	Entradas táctiles capacitivas físicas.
`SIM_TOUCH_UP_BUTTON_PIN` / `SIM_TOUCH_DOWN_BUTTON_PIN`	`uint8_t`, GPIO25 / GPIO26	Entradas de botones Wokwi usadas solo en modo simulación.
`OLED_SDA_PIN` / `OLED_SCL_PIN`	`uint8_t`, GPIO21 / GPIO22	Pines de bus I2C para la pantalla.

Resolución, umbrales e intervalos

Nombre	Tipo / valor	Propósito
`PWM_BITS`	`uint8_t`, 12	Configuración de resolución ADC/PWM.
`PWM_MAX`	`uint16_t`, `(1 << PWM_BITS) - 1`	Máximo de 12 bits: 4095.
`TOUCH_UP_THRESHOLD`	`uint16_t`, 950	Umbral táctil físico inicial; se debe calibrar.
`TOUCH_DOWN_THRESHOLD`	`uint16_t`, 850	Umbral táctil físico inicial; se debe calibrar.
`TOUCH_RELEASE_MARGIN`	`uint16_t`, 100	Margen de histéresis mientras un toque está retenido.
`EXTERNAL_LED_STEP`	`uint16_t`, `PWM_MAX / 20`	Un paso nominal de brillo del cinco por ciento.
`SENSOR_INTERVAL_MS`	`unsigned long`, 2000	Intervalo de muestreo periódico en el loop; `setup()` también hace una lectura inicial.
`DISPLAY_INTERVAL_MS`	`unsigned long`, 250	Frecuencia de redibujado OLED.
`THINGSPEAK_INTERVAL_MS`	`unsigned long`, 16000	Intervalo de intentos de escritura al canal.
`WIFI_RETRY_INTERVAL_MS`	`unsigned long`, 10000	Intervalo de intentos de reconexión.

Objetos y estado mutable

Nombre	Tipo / valor inicial	Propósito
`display`	`Adafruit_SH1106G(128, 64, &Wire, -1)`	Instancia del controlador OLED; `-1` indica que no se proporciona un pin de reset.
`dht`	`DHT(DHT_PIN, DHT22)`	Controlador del sensor DHT22.
`thingspeakClient`	`WiFiClient`	Cliente de red que se entrega a la biblioteca ThingSpeak.
`temperatureC` / `humidityPct`	`float`, `NAN`	Últimos valores válidos; quedan en NaN hasta una lectura completa exitosa.
`integratedPwm` / `externalPwm`	`uint16_t`, 0	Valores PWM actuales de 12 bits.
`touchUpLatched` / `touchDownLatched`	`bool`, false	Estado de bloqueo por flanco, para evitar cambios repetidos mientras se mantiene el toque.
`displayAvailable`	`bool`, false	Protege las funciones de dibujo si falla la inicialización OLED.
`otaStarted`	`bool`, false	Garantiza que la configuración OTA se ejecute una vez después de conectar Wi-Fi.
`lastSensorReadMs`	`unsigned long`, 0	Marca temporal del programa de muestreo DHT.
`lastDisplayMs`	`unsigned long`, 0	Marca temporal del programa de actualización de OLED.
`lastThingSpeakMs`	`unsigned long`, 0	Marca temporal del programa de escritura ThingSpeak.
`lastWiFiRetryMs`	`unsigned long`, 0	Marca temporal del programa de reconexión Wi-Fi.

Variables locales y parámetros de funciones

Nombre	Alcance / función	Significado
`newTemperature`, `newHumidity`	`updateSensorReadings()`	Resultados DHT22 nuevos guardados temporalmente para actualizar ambas globales juntas solo si las dos lecturas son válidas.
`rawPot`	`updatePotentiometerLed()`	Muestra ADC actual del potenciómetro antes de mapearla a PWM.
`upPressed`, `downPressed`	`updateTouchControls()`	Booleanos normalizados para los botones simulados o el estado táctil físico calibrado.
`upReading`, `downReading`	Modo físico de `updateTouchControls()`	Lecturas capacitivas sin procesar para umbrales/histéresis y diagnósticos serie.
`lastTouchReportMs`	Modo físico de `updateTouchControls()`, `static`	Conserva entre llamadas la marca temporal del diagnóstico táctil, que se informa una vez por segundo.
`externalPercent`	`drawStatus()` y `sendThingSpeakUpdate()`	Conversión local de `externalPwm` al porcentaje 0–100 mostrado/publicado. Son dos variables locales distintas con el mismo nombre.
`randomValue`	`sendThingSpeakUpdate()`	Valor aleatorio del campo 1; `random(100, 501)` produce del 100 al 500 inclusive.
`elapsedSeconds`	`sendThingSpeakUpdate()`	`millis()` convertido de milisegundos a segundos enteros para el campo 4.
`response`	`sendThingSpeakUpdate()`	Estado devuelto por `ThingSpeak.writeFields()`.

Nombre	Alcance / función	Significado
`error`	Callback `ArduinoOTA.onError`	Código de error OTA que se imprime por Serial.
`now`	`loop()`	Marca actual en milisegundos para programar el trabajo periódico.
`lastWiFiRetryMs`	`connectToWiFi()` y `loop()`	La marca global se establece al iniciar Wi-Fi y luego `loop()` la lee/actualiza para programar reconexiones; no existe una segunda marca local de reintento.

Funciones

Función	Comportamiento
`connectToWiFi()`	Selecciona el modo estación, inicia la asociación Wi-Fi con credenciales locales, registra la marca temporal de reintento e informa el nombre de red solicitado. No espera bloqueada hasta que se complete la asociación.
`startOtaWhenConnected()`	Retorna si Wi-Fi no está conectado o si OTA ya se inició. Establece el hostname `ESP32_TPI4K3`, registra callbacks serie de inicio/fin/error, inicia `ArduinoOTA`, marca el servicio como activo e informa la dirección local de la placa.
`updateSensorReadings()`	Lee temperatura y humedad. Si alguna lectura no es válida, informa el error y conserva los últimos valores conocidos; si no, actualiza ambas globales juntas.
`updatePotentiometerLed()`	Lee GPIO32, mapea el rango ADC `0..4095` a `0..PWM_MAX`, guarda el PWM del LED integrado y lo aplica a GPIO2.
`applyExternalLedBrightness()`	Aplica el valor PWM externo actual a GPIO23.
`updateTouchControls()`	Lee botones virtuales activos en nivel bajo en modo simulación o mediciones táctiles físicas. En modo físico informa las lecturas aproximadamente una vez por segundo; aplica umbral e histéresis de liberación; aumenta/disminuye el PWM externo una vez por nuevo flanco de pulsación; limita el valor al rango válido y actualiza la salida. Mantener un pad presionado solo cambia el brillo una vez hasta soltarlo.
`drawStatus()`	No hace nada si la OLED no está disponible. Si está disponible, muestra el título común, el PWM bruto integrado, el estado del sensor o el mensaje de espera, el porcentaje del LED externo y el estado/dirección de Wi-Fi.
`haveValidReadings()`	Devuelve true solo si temperatura y humedad no son NaN.
`sendThingSpeakUpdate()`	Omite la escritura si no hay Wi-Fi o lecturas válidas del sensor. Calcula los valores aleatorios, tiempo transcurrido y brillo; asigna los seis campos; llama a `ThingSpeak.writeFields()`; informa la respuesta y el éxito cuando el estado es 200.
`setup()`	Inicializa Serial y ADC/PWM de 12 bits, configura modos de entrada/salida, inicializa I2C y la OLED opcional, inicia DHT y clientes ThingSpeak, inicializa el generador aleatorio, inicia Wi-Fi y realiza las primeras actualizaciones de potenciómetro/LED/sensor/pantalla. La falla de OLED no es fatal.
`loop()`	Reintenta Wi-Fi según el intervalo; inicia OTA una vez conectado y la atiende en cada iteración posterior; muestrea continuamente potenciómetro y touch; programa lecturas del sensor, refresco de pantalla y escrituras ThingSpeak usando intervalos transcurridos de `millis()`.

Inicio ordenado (`setup()`)

1. Inicia Serial a 115200 y configura ADC a 12 bits y PWM a 12 bits por pin para GPIO2 y GPIO23.
2. Configura el potenciómetro como entrada y ambos LED como salidas. Solo en modo de simulación Wokwi, configura GPIO25/26 como entradas con pull-up.

3. Inicia I2C en GPIO21/GPIO22 e inicializa la pantalla en la dirección `0x3C`. Si no se detecta, informa el mensaje y continúa sin pantalla; si está disponible, configura tamaño/color de texto.
4. Inicia el controlador DHT22 y el cliente ThingSpeak, inicializa el generador pseudoaleatorio e inicia Wi-Fi en modo estación.
5. Actualiza inicialmente potenciómetro a LED, establece la salida del LED externo en su nivel actual cero, intenta una lectura inicial del DHT y dibuja el estado inicial si la pantalla está disponible.

Trabajo repetido (`loop()`)

1. Si se perdió la conexión y ya transcurrió el intervalo de reintento, llama a `WiFi.reconnect()` y actualiza la marca temporal.
2. Inicia ArduinoOTA cuando el enlace Wi-Fi está listo. Una vez iniciado, llama a `ArduinoOTA.handle()` en cada vuelta para procesar solicitudes OTA.
3. Mapea continuamente el potenciómetro al LED integrado y comprueba los controles táctiles para detectar flancos.
4. Cada 2 segundos actualiza las lecturas DHT. Cada 250 ms redibuja la OLED. Cada 16 segundos intenta publicar los seis campos.

El loop usa comparaciones de tiempo transcurrido en lugar de una demora larga, lo que permite atender OTA y actualizar controles entre tareas periódicas. Aun así, las llamadas individuales a red/bibliotecas pueden demorar.

7. Recorrido del Sketch 2 — lector y contador de pulsaciones

Fuente: `sketch_lectura_grupo_XX/sketch_lectura_grupo_XX.ino`

Estado y constantes globales

Configuración de red, hardware y tiempos

Nombre	Tipo / valor inicial	Propósito
`WIFI_SSID` / `WIFI_PASSWORD`	`const char *`	Credenciales Wi-Fi locales; redactalas y mantenelas privadas.
`THINGSPEAK_CHANNEL_ID`	`const char *`, identificador del canal	Canal público que consulta el lector.
`THINGSPEAK_READ_API_KEY`	`const char *` opcional, comentado	Habilitalo solo si el canal pasa a ser privado; mantené el valor privado.
`BUTTON_PIN`	`uint8_t`, GPIO19	Pulsador del encoder, supuesto activo en LOW.
`OLED_SDA_PIN` / `OLED_SCL_PIN`	`uint8_t`, GPIO21 / GPIO22	Pines I2C de pantalla.
`READ_INTERVAL_MS`	`unsigned long`, 16000	Intervalo de consulta de la última entrada del canal.
`PAGE_INTERVAL_MS`	`unsigned long`, 5000	Cambia cada cinco segundos el grupo de campos mostrado.
`DISPLAY_REFRESH_INTERVAL_MS`	`unsigned long`, 250	Frecuencia de redibujado.
`BUTTON_DEBOUNCE_US`	`uint32_t`, 200000	Ignora flancos aceptados que ocurran con menos de 200 ms de diferencia.

Pantalla, interrupción y estado de ejecución

Nombre	Tipo / valor inicial	Propósito
`display`	`Adafruit_SH1106G(128, 64, &Wire, -1)`	Controlador OLED 128×64.

Nombre	Tipo / valor inicial	Propósito
`fieldValues[6]`	Arreglo `String` inicializado en `"--"`	Valores recientes formateados; muestra espera hasta recibir datos.
`fieldLabels[6]`	Arreglo `const char *`	Etiquetas de pantalla en español/breves en orden de campo.
`buttonPressCount`	`volatile uint32_t`, 0	Contador de pulsaciones actualizado por interrupción. `volatile` evita que el compilador lo mantenga indebidamente en caché; la sección crítica permite accederlo de forma segura.
`lastButtonInterruptUs`	`volatile uint32_t`, 0	Hora de la última pulsación aceptada, en microsegundos.
`buttonMux`	Inicializador `portMUX_TYPE`	Bloqueo de sección crítica compartido entre ISR y código normal.
`lastReadMs`	`unsigned long`, 0	Última consulta programada al canal.
`lastPageChangeMs`	`unsigned long`, 0	Último cambio de página.
`lastDisplayMs`	`unsigned long`, 0	Último redibujado OLED.
`currentPage`	`uint8_t`, 0	Página activa: 0 o 1.
`haveChannelData`	`bool`, false	Indica que se aceptó una respuesta completa de seis campos.
`displayAvailable`	`bool`, false	Protege el dibujo OLED si falla la inicialización.

Variables locales y parámetros de funciones

Nombre	Alcance / función	Significado
`nowUs`	`onButtonPressed()`	Marca en microsegundos capturada en el contexto de interrupción para la temporización antirrebote.
`count`	`getButtonPressCount()`	Copia estable del contador actualizado por interrupción mientras se mantiene la sección crítica.
`timeoutMs`	`connectToWiFi(timeoutMs)`	Tiempo máximo de espera inicial de Wi-Fi solicitado por quien llama.
`startedAt`	`connectToWiFi()`	Marca en milisegundos usada junto con `timeoutMs` para finalizar la espera inicial.
`payload`	`parseChannelResponse(payload)`	Texto de respuesta HTTP que se entrega al analizador JSON.
`doc`	`parseChannelResponse()`	Documento ArduinoJson acotado que almacena la respuesta analizada.
`error`	`parseChannelResponse()`	Resultado de deserialización; un error distinto de cero impide reemplazar los datos.
`keys`	`parseChannelResponse()`	Seis nombres de propiedad esperados del JSON, `field1` a `field6`.
`i`	`parseChannelResponse()`	Índice de bucle para validar los seis campos y copiarlos luego a `fieldValues`.
`http`	`readThingSpeak()`	Cliente HTTP de vida corta, cerrado explícitamente mediante `http.end()`.
`url`	`readThingSpeak()`	URL de solicitud del último feed, opcionalmente ampliada con una clave de lectura privada.
`httpCode`	`readThingSpeak()`	Estado que devuelve la solicitud GET.

Nombre	Alcance / función	Significado
`payload`	`readThingSpeak()`	Texto del cuerpo devuelto por `http.getString()`; es local a esta función y distinto del parámetro del analizador.
`firstField`	`drawCurrentPage()`	Índice inicial (0 o 3) de la página activa de tres campos.
`i`	`drawCurrentPage()`	Índice de campo desde `firstField` hasta los siguientes tres valores.
`now`	`loop()`	Marca actual en milisegundos para temporizadores de lectura/página/pantalla.
`lastReconnectMs`	`loop()`, variable local `static`	Conserva entre llamadas al loop la hora del último pedido de reconexión Wi-Fi.

Funciones

Función	Comportamiento
`onButtonPressed()`	Manejador de interrupción `IRAM_ATTR`. Lee microsegundos, entra en una sección crítica segura para ISR, acepta el flanco solo si transcurrió el intervalo antirrebote, aumenta el contador, registra el tiempo aceptado y libera el bloqueo.
`getButtonPressCount()`	Copia el contador actualizado por ISR dentro de la sección crítica normal y devuelve una captura consistente.
`connectToWiFi(timeoutMs)`	Inicia la asociación en modo estación y espera en intervalos de 100 ms hasta el tiempo solicitado. Informa éxito y dirección o vencimiento del tiempo. El loop principal vuelve a intentar tras una desconexión posterior.
`parseChannelResponse(payload)`	Deserializa JSON en un documento acotado de 1024 bytes, verifica que existan las seis claves esperadas, convierte cada valor a texto y recién entonces marca los datos del canal como disponibles. Rechaza datos malformados o incompletos sin reemplazar parcialmente el conjunto mostrado.
`readThingSpeak()`	Requiere Wi-Fi; si está desconectado, informa y solicita reconexión. Construye la URL del último feed, permite opcionalmente una clave de lectura de canal privado, establece timeout HTTP de 5 segundos, ejecuta GET, analiza respuestas HTTP 200, informa errores y cierra el cliente HTTP con `end()`.
`drawCurrentPage()`	Retorna si la pantalla no está disponible. Muestra el título común y la página, representa tres de los seis campos según la página seleccionada y muestra el contador actual. Antes de la primera lectura correcta, los valores muestran `waiting`.
`setup()`	Inicia Serial, configura GPIO19 con pull-up y asocia una interrupción por flanco descendente, inicializa I2C/OLED opcional, conecta a Wi-Fi con espera de 15 segundos, solicita inmediatamente los datos del canal e inicializa los temporizadores de lectura/página.
`loop()`	Intenta reconectar Wi-Fi cada 10 segundos mientras está desconectado; lee ThingSpeak cada 16 segundos; cambia de página cada 5 segundos; redibuja cada 250 ms; espera 20 ms por vuelta para reducir la actividad ocupada.

Inicio ordenado y comportamiento repetido

1. **Primero el pulsador:** inicia Serial, habilita el pull-up interno de GPIO19 y asocia `onButtonPressed()` al flanco descendente. El código supone que el pulsador conecta el pin a masa cuando se presiona.
2. **Pantalla opcional:** inicializa I2C y la OLED en `0x3C`. Si no está presente, lo informa, pero no detiene Wi-Fi ni la consulta del canal.

3. **Lectura inicial:** espera hasta 15 segundos por Wi-Fi y luego consulta la última entrada de ThingSpeak. La rutina setup fija los temporizadores de consulta/página después del intento inicial.
4. **Loop continuo:** reconecta una conexión Wi-Fi perdida cada 10 segundos, consulta el canal según el intervalo de 16 segundos, alterna entre campos 1–3 y 4–6 cada cinco segundos y redibuja/cuenta cada 250 ms.

La pantalla no muestra el giro A/B del encoder porque no está implementado. Solo el pulsador incrementa el contador. El contador se reinicia al reiniciar la placa y no se envía a ThingSpeak.

8. Variante del lector OTA y flujos de carga

Fuente: `sketch_lectura_grupo_XX_ota/sketch_lectura_grupo_XX_ota.ino`

Esta variante conserva las constantes, arreglos de campos, lógica de interrupción/antirrebote, analizador JSON, lector HTTP, páginas de pantalla y programación del lector. Sus credenciales son marcadores seguros. Diferencias frente al lector normal:

Adición / diferencia	Efecto
Incluye `ArduinoOTA.h`	Incorpora el servicio OTA del core ESP32.
`OTA_HOSTNAME`	Establece el hostname visible en la red como `ESP32_TPI4K3`, igual que en el Sketch 1.
`otaStarted`	Indica si se inicializó el servicio OTA.
`startArduinoOTA()`	Registra callbacks de inicio/fin/error OTA, fija el hostname, inicia el listener OTA una vez conectado e informa hostname/dirección.
`connectToWiFi()`	Cuando la conexión inicial tiene éxito, inicia ArduinoOTA antes de retornar.
`loop()`	Atiende `ArduinoOTA.handle()` cuando está activo e inicia OTA tras una reconexión Wi-Fi posterior si todavía no se había iniciado.
Detalle del log Wi-Fi	La variante conserva el log de dirección/estado local, pero no imprime el SSID en su rutina de conexión.

Los nombres propios de esta variante son `OTA_HOSTNAME` (hostname constante compartido con Sketch 1), `otaStarted` (indica si se inicializó el servicio OTA) y `error` (parámetro del código de error del callback OTA). También usa `timeoutMs` y `startedAt` para la espera inicial acotada de Wi-Fi y el mismo local `now` del lector. Los demás locales/parámetros son los del lector descritos en la sección 7; esta variante no agrega otro conjunto de funciones o estado del lector.

Instalación inicial y dirección habitual

1. Ingresá las credenciales Wi-Fi locales en el Sketch 1. Mantené privada la clave de escritura; el transporte OTA no la necesita.
2. Conectá la ESP32 por USB y cargá el Sketch 1.
3. Abrí el Monitor Serie a 115200 y esperá el mensaje de disponibilidad OTA cuando se conecte Wi-Fi.
4. Abrí el sketch lector original, seleccioná en el IDE el puerto de red detectado `ESP32_TPI4K3` y cargá el Sketch 2 por OTA.

El usuario confirmó la dirección física Sketch 1 por USB → Sketch 2 original por OTA. No tomes ese resultado como prueba de la dirección de retorno.

Recuperación de vuelta al Sketch 1

1. Asegurate de que la variante del lector OTA tenga configuradas credenciales Wi-Fi locales privadas y compilala para el mismo objetivo ESP32.
2. Cargá la variante del lector OTA en el dispositivo mediante la vía inicial disponible (USB es la ruta de recuperación segura si el firmware actual no tiene servicio OTA).

3. Esperá el mensaje serie `OTA ready`. La variante debe estar en una red a la que pueda llegar la laptop.
4. Seleccioná el puerto de red detectado de la ESP32 y cargá el Sketch 1 por OTA.
5. Confirmá que el emisor inicia, se conecta e informa que OTA está listo; después verificá ThingSpeak y los periféricos por separado.

Salvedad de evidencia: la variante de lector OTA compiló, pero no se probó la carga física de esa variante → Sketch 1. No informes una ida y vuelta OTA verificada hasta hacer esa prueba.

9. Firewall Linux, detección y diagnóstico de mDNS

OTA es una operación de red local, no simplemente una carga a la nube. El IDE necesita detectar la placa, enviar la carga y recibir el tráfico de respuesta del protocolo OTA. En particular, puede requerirse una **conexión TCP inversa desde la ESP32 hacia la computadora donde corre el IDE**. El firewall del host podría permitir la detección inicial y bloquear esa conexión de respuesta.

El problema de firewalld informado por el usuario se resolvió permitiendo el camino de respuesta OTA a través del firewall del host. Esta guía omite deliberadamente el nombre de red, la dirección y la regla específica de esa máquina privada. Aplicá cambios de firewall únicamente a la red local de confianza y limitá las reglas tanto como permitan los puertos observados y la red de origen.

Secuencia de diagnóstico

1. **Confirmá que esté listo:** la placa debe estar conectada a Wi-Fi e informar `OTA ready`; verificá que el IDE y la placa estén en la misma LAN/VLAN accesible.
2. **Revisá el aislamiento:** la red Wi-Fi de invitados, el aislamiento de clientes/AP, las ACL de VLAN, el enrutamiento VPN o una subred separada pueden bloquear la detección y el tráfico directo. Usá una red que permita comunicación entre dispositivos.
3. **Revisá mDNS:** la detección de puertos de red de Arduino IDE suele depender de multicast mDNS local (UDP 5353). Verificá que el multicast no esté bloqueado entre la laptop y la placa; en Linux también puede depender de un respondedor mDNS como Avahi. Que no aparezca un hostname indica un problema de detección, no prueba que falte el firmware.
4. **Revisá la ruta OTA:** ArduinoOTA usa su puerto de servicio/control (comúnmente 3232 en ESP32 Arduino) y el cargador puede necesitar la conexión TCP inversa dinámica al host del IDE. Inspeccioná la zona activa y los registros del firewall durante un intento. Permití solo el tráfico local necesario; evitá aperturas amplias de zona pública o de todos los puertos.
5. **Revisá la zona/interfaz firewalld:** la interfaz Wi-Fi podría estar asociada a una zona distinta de la esperada. Creá una regla acotada y reversible para la LAN de confianza y la ruta de respuesta observada; luego volvé a probar. No copies a ciegas la regla de otra máquina.
6. **Probá una dirección local directa solo como diagnóstico:** si conocés la dirección de la placa por Serial, verificá la conectividad laptop-placa y la ruta de retorno. Que responda un ping no demuestra que los puertos del protocolo OTA estén abiertos.
7. **Recuperá por USB:** si no aparece ningún punto OTA, usá USB para instalar un sketch conocido y funcional; después revisá Wi-Fi, hostname, detección y firewall antes de volver a intentar.

mDNS y la detección del puerto de red no son lo mismo que el acceso a ThingSpeak. La ESP32 puede leer/escribir ThingSpeak correctamente aunque la laptop no pueda detectar la placa para OTA.

Ejemplo de permiso acotado en firewalld

Primero revisá qué zona de firewalld tiene asignada la interfaz Wi-Fi; luego reemplazá `<WIFI_ZONE>` y `<ESP32_IP>` por los valores de la red local de confianza. Esto refleja el enfoque de diagnóstico que funcionó sin exponer la dirección privada de una persona:

```
sudo firewall-cmd --get-active-zones
sudo firewall-cmd --zone="<WIFI_ZONE>" --add-service=mdns
sudo firewall-cmd --zone="<WIFI_ZONE>" \
  --add-rich-rule='rule family="ipv4" source address="<ESP32_IP>" port port="10000-60000"
protocol="tcp" accept'
```

La primera regla de ejecución permite la detección mDNS. La regla rich permite la conexión TCP inversa de la ESP32 al host únicamente desde la dirección de ese dispositivo; el cargador elige dinámicamente el puerto de respuesta del host. No abras este rango a cualquier origen ni desactives el firewall. Una reserva DHCP puede mantener estable la dirección de la ESP32. Después de confirmar la carga, persistí solo las reglas necesarias mediante `--permanent` y recargá firewalld:

```
sudo firewall-cmd --permanent --zone="<WIFI_ZONE>" --add-service=mdns
sudo firewall-cmd --permanent --zone="<WIFI_ZONE>" \
  --add-rich-rule='rule family="ipv4" source address="<ESP32_IP>" port port="10000-60000"
protocol="tcp" accept'
sudo firewall-cmd --reload
```

Límite de seguridad

Estos sketches no configuran contraseña OTA ni comprobación de autenticación. Cualquier persona que pueda alcanzar el servicio OTA en la LAN de confianza podría intentar cargar firmware. Usá únicamente una red que controles, no expongas el puerto OTA a Internet y no confundas el requisito OTA del TPI con un diseño seguro de actualización remota. La autenticación, el firmware firmado, la reversión y una política de actualización de producción están fuera de esta implementación académica.

10. Wokwi frente a la verificación física

Comportamiento	Qué ayuda a verificar Wokwi	Qué requiere hardware físico / red real
Compilación y ejecución básica del sketch	Sí, con objetivo ESP32 y las bibliotecas listadas	También hay que compilar con la versión del core prevista para la placa física antes de la entrega final.
Distribución y texto OLED	Distribución aproximada con el modelo de simulación SSD1306	Comportamiento exacto con el controlador/pantalla SH1106 del curso y su cableado.
Lecturas DHT y valores mostrados	Los controles virtuales DHT permiten ejercitar el camino de datos/pantalla	Precisión del sensor, cableado, temporización y comportamiento del dispositivo real.
Respuesta del potenciómetro y LED	El potenciómetro y los LED virtuales prueban los mapeos	Rango eléctrico, identidad del LED integrado, polaridad y limitación de corriente del LED externo.
Aumento/disminución táctil	Emulación mediante pulsadores en modo Wokwi	Mediciones capacitivas reales, umbrales, cableado de electrodos y compatibilidad de GPIO13/GPIO4.
Contador de pulsador del encoder	El encoder virtual puede ejercitar la lógica de GPIO19	Cableado real del encoder del curso, rebote del pulsador y uso del contador. El giro A/B no está implementado.
Publicación/lectura ThingSpeak	Puede ejercitar el código de red si están configurados el acceso de red Wokwi y los parámetros privados	Credenciales reales, configuración del canal y comportamiento de red real. Un resultado Wokwi no sustituye una prueba de la ruta de hardware.

Comportamiento	Qué ayuda a verificar Wokwi	Qué requiere hardware físico / red real
Detección/carga OTA	No es una prueba OTA física	Debe usarse la ESP32 real, el host del IDE, la misma LAN accesible y el sketch activo correspondiente.

El diagrama usa una pieza de simulación SSD1306, mientras que las fuentes apuntan a un controlador de la familia SH1106. Sirve para revisar el texto y la distribución general, no para demostrar que la OLED física se comporta igual. Los pulsadores Wokwi en GPIO25/GPIO26 emulan la entrada táctil; el emisor físico debe usar touch GPIO13/GPIO4.

11. Verificación y nivel de confianza

La evidencia se registra por categoría para no confundir una compilación, una simulación y una prueba física.

Categoría de evidencia	Estado observado	Qué demuestra / qué no demuestra
Comportamiento informado en Wokwi	El usuario informó que el Sketch 1 funcionaba con pantalla/datos del sensor, potenciómetro, emulación táctil con botones, PWM de LED y actualizaciones ThingSpeak; después informó que ambos sketches principales funcionaban en Wokwi.	Confirma el comportamiento de simulación informado en esos momentos. No demuestra calibración táctil física, comportamiento exacto de SH1106 ni OTA física. Las ediciones posteriores de modo sin pantalla/recuperación también tienen comprobaciones de código/compilación, no se afirma una regresión Wokwi completa reciente.
Comprobaciones estáticas/de código	Los registros de tareas previas documentan verificaciones de configuración a 12 bits, mapeo de campos, temporizadores, inicio/manejador OTA, análisis/páginas/botón del lector, manejo de pantalla ausente y pines del diagrama.	Revisan la estructura del código y ciertos invariantes; no prueban el comportamiento eléctrico o de red en ejecución.
Compilación	La variante del lector con OTA habilitado compiló con el compilador ESP32 incluido en el IDE.	Establece que esa variante compiló en ese entorno; no demuestra carga, detección OTA, acceso real al canal ni periféricos conectados.
Prueba OTA física	El usuario confirmó Sketch 1 por USB → Sketch 2 original por OTA.	Demuestra esa dirección en la configuración del usuario. No demuestra la carga de retorno desde la variante de lector OTA.
Pendiente / aún no verificado	Carga física de retorno desde la variante de lector OTA → Sketch 1; comprobación física integral del DHT22, OLED, potenciómetro, LED, umbrales táctiles y encoder; artefactos de entrega de grupo.	Hay que completar y registrar estas pruebas antes de afirmar que el hardware o la entrega están completamente listos. La lista de tareas identifica el trabajo físico pendiente y la dependencia del grupo/integrantes.

12. Casos límite y comportamiento conocido

- **Sin OLED conectada:** se informa el fallo de inicialización; la red y el funcionamiento sin pantalla continúan. En este modo no se puede comprobar la salida OLED.
- **Sin DHT22 o con lectura incorrecta:** el emisor conserva el último par válido; antes de obtener un par válido omite las escrituras ThingSpeak. Por eso puede hacerse una prueba OTA con la placa sola mientras se omite la publicación.

- **La sensibilidad táctil varía:** los umbrales derivan de un ejemplo y dependen de la placa. Leé los valores reales en reposo y al tocar antes de ajustar; no supongas que el botón virtual Wokwi valida el hardware táctil.
- **Diferencias en el LED integrado:** GPIO2 es un supuesto explícito. Una placa puede tener el LED en otro pin, polaridad diferente o no tenerlo. Verificalo en la placa física del curso.
- **Respuesta ThingSpeak parcial/malformada:** el lector rechaza JSON inválido o la falta de cualquiera de los seis campos y conserva los valores mostrados anteriormente. Si nunca obtuvo una respuesta completa, la pantalla muestra valores de espera.
- **Cambio de privacidad del canal:** para leer un canal público no hace falta clave; uno privado sí la requiere. Mantené privada cualquier clave de lectura y habilitá el parámetro comentado solo en una copia local.
- **Sin red:** ambos roles intentan reconectarse periódicamente; las solicitudes de datos pueden omitirse o devolver errores HTTP. Serial es el diagnóstico principal cuando no hay pantalla.
- **Rebote del pulsador:** el antirrebote de 200 ms descarta flancos rápidos. Es una ventana fija por software y puede requerir verificación con el pulsador real.
- **Persistencia del contador:** el contador se reinicia al encender/reiniciar y eventualmente puede desbordarse como contador sin signo de 32 bits. Solo reside en RAM y no es un campo ThingSpeak.
- **Desbordamiento del temporizador:** `millis()` y `micros()` de Arduino se desbordan. Las comprobaciones periódicas usan resta sin signo de tiempo transcurrido; el campo 4 de segundos transcurridos vuelve a cero al reiniciar.
- **Alcance OTA:** conectarse a Wi-Fi no garantiza la detección mDNS ni la ruta TCP inversa. El firewall, el aislamiento, los filtros multicast y el enrutamiento pueden impedir la carga.
- **Sin autenticación OTA:** estas fuentes no configuran una contraseña OTA. Mantené la placa en una LAN de confianza y nunca expongas OTA a redes no confiables.
- **Nombres de sketches:** `XX` es un marcador para el número de grupo. Los nombres finales de entrega deben actualizarse de manera consistente, conservando la relación entre cada archivo fuente y su carpeta como lo exija la materia.

13. Lista de verificación de compilación, prueba y entrega

Antes de compartir esta guía o cualquier artefacto del proyecto

☐ Reemplazá todas las credenciales reales de Wi-Fi y claves de escritura/lectura en las fuentes locales por marcadores neutros antes de crear una copia compartible.

☐ Revisá capturas de pantalla, logs serie, configuración del proyecto Wokwi y archivos de texto para detectar secretos, nombres de red privados y direcciones privadas.

☐ Mantené fuera de repositorios y chats públicos cualquier archivo privado de entrega que contenga la clave de escritura requerida.

☐ Eliminá o rotá las credenciales si se expusieron fuera de su entorno privado previsto.

Firmware y hardware

☐ Instalá/seleccioná el core Arduino ESP32 previsto y todas las bibliotecas externas.

☐ Compilá los dos sketches principales y la variante opcional del lector OTA para la placa objetivo.

☐ Confirmá el pin del LED integrado en la placa física; GPIO2 es solo el supuesto documentado.

☐ Verificá dirección/cableado de OLED y comportamiento SH1106 real; confirmá que el inicio sin pantalla siga siendo posible.

☐ Medí los valores de reposo y al tocar en GPIO13/GPIO4, calibrá umbrales y comprobá un paso de brillo por pulsación.

☐ Verificá DHT22, PWM del potenciómetro al LED integrado, limitación de corriente/PWM del LED externo y pulsador encoder GPIO19.

☐ Confirmá los seis campos ThingSpeak, la frecuencia de actualización de 16 segundos y la lectura de la última entrada.

☐ Probá y registrá Sketch 1 por USB → Sketch 2 original por OTA.

☐ Probá y registrá el camino de retorno todavía no verificado usando la variante lectora OTA → Sketch 1.

☐ Registrá qué resultados corresponden a simulación, comprobaciones estáticas/compilación y pruebas físicas.

Entrega de la materia

☐ Reemplazá `XX` en los nombres de ambos sketches requeridos por el número de grupo real.

☐ Creá el archivo de texto de integrantes con apellido, nombre y legajo de cada persona.

☐ Incluí el ID del canal y la clave de escritura requerida solo en el artefacto privado de entrega, no en una copia pública.

☐ Confirmá que los dos archivos principales entregados sean el emisor y el lector original; incluí la variante de recuperación solo si corresponde a la entrega de la materia.

☐ Confirmá que cada integrante esté preparado para la evaluación y que la versión UV entregada sea la versión probada.

14. Fuentes y referencias

Artefacto	Propósito
`TPI 4K3.pdf`	Requisitos originales, archivos solicitados y contexto de evaluación/entrega.
`sketch_envio_grupo_XX/sketch_envio_grupo_XX.ino`	Fuente principal del emisor/controlador. Antes de compartir, sanitizá los valores de credenciales locales.
`sketch_lectura_grupo_XX/sketch_lectura_grupo_XX.ino`	Fuente principal del lector ThingSpeak. Antes de compartir, sanitizá sus credenciales Wi-Fi locales.
`sketch_lectura_grupo_XX_ota/sketch_lectura_grupo_XX_ota.ino`	Lector opcional con recuperación ArduinoOTA y marcadores neutros de credenciales.
`wokwi/README.md`	Configuración Wokwi, salvedades de simulación frente a hardware e instrucciones de prueba OTA.
`wokwi/libraries.txt`	Lista de bibliotecas externas Wokwi/Arduino.
`wokwi/diagram.json`	Circuito ESP32 virtual, endpoints de pines y componentes.
`odd/tasks/tpi-4k3-esp32-thingspeak-ota.md`	Requisitos/tareas y registro cronológico de evidencia; sirve para contexto, pero no reemplaza el PDF del trabajo.

Notas de referencia

- Las asignaciones de pines se documentan en las fuentes actuales y el mapa de pines del proyecto. GPIO2 se declara explícitamente como supuesto del ejemplo del docente para el LED integrado.
- Se conserva el período de muestreo de dos segundos del DHT22 como frecuencia segura específica del dispositivo; la actualización de pantalla es independiente y más rápida.
- ArduinoOTA y las API de red provienen del core Arduino ESP32. La disponibilidad del puerto OTA depende del comportamiento del cargador de la versión de core/IDE elegida y de la red/firewall

local.

- Esta es una implementación educativa, no un diseño de actualización remota reforzado para producción.